
Algorithm 1: ParaFlow: Parallel Sampling within a Sliding Window

Input : Initial noise x_{t_0} , total steps N , timesteps $\{t_i\}_{i=0}^N$, tolerance δ , window size p .

Output : Final sample $\hat{x}_{t_N}^{(K)}$.

Initialize $\{\hat{x}_{t_i}^{(0)} = x_{t_0}, i = 0, \dots, p-1\}$.

$i \leftarrow 0, k \leftarrow 0;$

while $i < N$ **do**

 Compute vector field $v_j^{(k)} = v(\hat{x}_{t_{i+j}}^{(k)}, t_{i+j}, \theta)$ in parallel for $j \in \{0, \dots, p-1\}$.

 Update temporary states $F_j \leftarrow \hat{x}_{t_i}^{(k)} + \sum_{l=0}^j (t_{i+l+1} - t_{i+l}) v_l^{(k)}$ in parallel for $j \in \{0, \dots, p-1\}$.

 Update the state $\hat{x}_{t_{i+j+1}}^{(k+1)} \leftarrow F_j$ in parallel for $j \in \{0, \dots, p-1\}$.

$stride \leftarrow$ number of converged states based on $\frac{1}{D} \|\hat{x}_{t_{i+j+1}}^{(k+1)} - \hat{x}_{t_{i+j+1}}^{(k)}\|^2 \leq \delta^2$ for $j \in \{0, \dots, p-1\}$.

 Initialize new states outside current window: $\hat{x}_{t_{i+p+j}}^{(k+1)} \leftarrow \hat{x}_{t_{i+p}}^{(k+1)}$ for $j \in \{1, \dots, stride\}$.

$i \leftarrow i + stride, \quad k \leftarrow k + 1, \quad p \leftarrow \min(p, N - i).$

Return: $\hat{x}_{t_N}^{(K)}$
